



INGENIERIA EN DESARROLLO Y GESTION DE SOFTWARE

SEGG-U1-P1

Guerrero Jiménez Evelyn Regina

2023371194

Grupo: IDGS17

Materia:

Seguridad en el Desarrollo de Aplicaciones

Profesor:

Gerardo Ramírez Villarreal

GUIA DE LA PRACTICA

SEGG-U1-P1 — OWASP Top 10: Investigacion y analisis de vulnerabilidades

Unidad 1 | Navegador + documento | 4 hrs

Nombre del alumno:

Guerrero_Jiménez_Evelyn_Regina_____

No. de control:

__2023371194_____

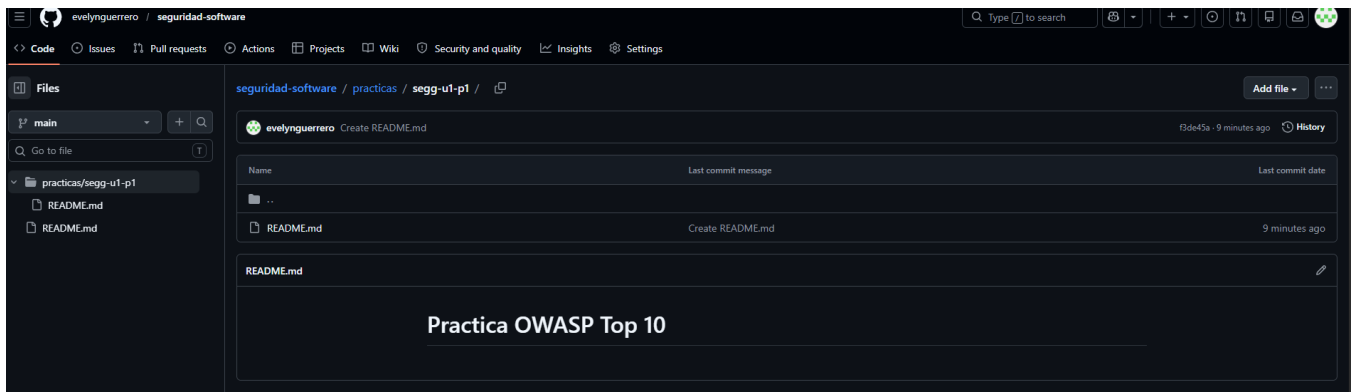
Esta guía contiene las instrucciones paso a paso para realizar la practica.
Lee completamente antes de comenzar. El entregable se describe al final del documento.
Fecha limite de entrega: semana 1-2 del cuatrimestre.

Objetivo

Investigar las 10 vulnerabilidades mas criticas segun OWASP Top 10 2023, comprender los mecanismos de ataque y su impacto en la seguridad de aplicaciones, y documentar casos reales de cada tipo de vulnerabilidad con ejemplos de codigo vulnerable.

Herramientas necesarias

- * Navegador web (Chrome, Firefox o Edge)
- * OWASP Top 10 2023: <https://owasp.org/Top10/>
- * Documento procesador de palabras para el reporte
- * GitHub para versionar el documento



Antecedentes

El OWASP Top 10 es una lista de las 10 categorias mas criticas de riesgos de seguridad en aplicaciones web, basada en datos de organizaciones de seguridad, proveedores y estudios independientes. Estudiar el OWASP Top 10 es fundamental para todo desarrollador, ya que proporciona una comprension de los riesgos mas comunes y como mitigarlos. Esta practica establece la base conceptual para todas las practicas de seguridad del curso.

Instrucciones

Sigue los pasos en el orden indicado. No saltes pasos aunque creas que son obvios.

#	SO	Instruccion / Comando
1	Ambos	Abre la pagina oficial de OWASP Top 10 2023 y lee la introduccion general.
2	Ambos	Para cada una de las 10 vulnerabilidades (A01 a A10), documenta en una tabla: nombre, descripcion corta, ejemplo de ataque, impacto potencial y metodo de mitigation.
3	Ambos	Investiga al menos 2 casos reales de cada vulnerabilidad (puedes buscar en: cvedetails.com, securityfocus.com o articulos de noticias tecnologicas). Documenta el nombre de la vulnerabilidad, la empresa afectada, el año, y como fue explotada.
4	Ambos	Para al menos 3 vulnerabilidades, incluye codigo vulnerable EN PSEUDOCODIGO O JAVASCRIPT y como deberia escribirse de forma segura. Ejemplo: SQL Injection vulnerable vs parametrized query.
5	Ambos	Crea un ranking personal de las 5 vulnerabilidades que consideres mas criticas segun tu criterio (pueden coincidir o no con el ranking de OWASP). Justifica cada seleccion en 2-3 lineas.
6	Ambos	Redacta una conclusion final de minimo 200 palabras sobre la importancia de conocer el OWASP Top 10 como desarrollador y como aplicarias este conocimiento en tu practica profesional.

Criterio de seguridad:

Cuando investigues casos reales, documenta solo informacion publica disponible en reportes de seguridad oficiales. Nunca busques o reproduzas exploits activos en sistemas que no sean de prueba. La intención es aprender, no causar dano.

Entregables

Para acreditar esta practica debes entregar TODOS los siguientes elementos:

- * Tabla comparativa de las 10 vulnerabilidades OWASP Top 10 2023 con descripcion, ejemplo de ataque, impacto y mitigacion
- * Al menos 2 casos reales por vulnerabilidad con contexto (empresa, año, como fue explotada)
- *Codigo vulnerable vs seguro para minimo 3 vulnerabilidades (pseudocodigo o JavaScript)
- * Ranking personal de las 5 vulnerabilidades mas criticas con justificacion
- * Conclusion de minimo 200 palabras sobre la importancia del OWASP Top 10

Sube tus entregables a tu repositorio de GitHub en la carpeta: practicas/segg-u1-p1/
Incluye reportes y capturas de pantalla como evidencia del trabajo realizado.
La practica se entrega en la semana 1-2 del cuatrimestre.

1-Abre la pagina oficial de OWASP Top 10 2023 y lee la introduccion general.

Introducción



OWASP top 10: son las 10 vulnerabilidades criticas más comunes en paginas web, owasp es un proyecto abierto para seguridad en la web, donde expertos comparten su conocimiento para poder mejorar la seguridad en la web, la owasp cada cierto tiempo actualiza una lista de las top 10

vulnerabilidades de seguridad para que los desarrolladores estén concientes de las amenazas así como poder mitigarlas y proteger sus desarrollos, dentro se pueden encontrar vulnerabilidades como inyección SQL , Cross site scripting, deserialización insegura, errores de autenticación.

2-Para cada una de las 10 vulnerabilidades (A01 a A10), documenta en una tabla: nombre, descripcion corta, ejemplo de ataque, impacto potencial y metodo de mitigation.

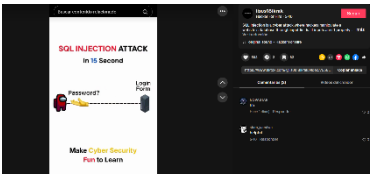
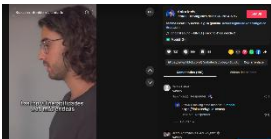
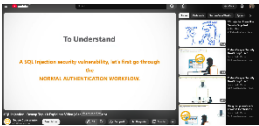
ID	Vulnerabilidad	Descripción corta	Ejemplo de ataque	Impacto potencial	Método de mitigación
A01	Broken Access Control	Sucede cuando los usuarios pueden acceder a funciones o información restringida sin autorización adecuada.	Un usuario modifica el ID de una URL para visualizar datos de otra cuenta.	Robo, modificación o eliminación de información sensible.	Implementar controles de acceso y validación de permisos en cada solicitud.
A02	Cryptographic Failures	Ocurre cuando los datos sensibles no son protegidos correctamente mediante cifrado.	Contraseñas almacenadas en texto plano dentro de la base de datos.	Filtración de datos personales, financieros o credenciales.	Utilizar cifrado seguro, HTTPS y algoritmos criptográficos actualizados.
A03	Injection	Se produce cuando una aplicación ejecuta datos maliciosos enviados por el atacante.	Inyección SQL en formularios de inicio de sesión.	Acceso no autorizado a bases de datos y robo de información.	Validar entradas y utilizar consultas parametrizadas.
A04	Insecure Design	Vulnerabilidad causada por un diseño inseguro desde el desarrollo inicial del sistema.	Aplicación sin límites de intentos de inicio de sesión.	Facilita ataques y compromete la seguridad general del sistema.	Aplicar principios de desarrollo seguro desde la fase de diseño.
A05	Security Misconfiguration	Configuraciones incorrectas o inseguras en servidores, aplicaciones o servicios.	Uso de contraseñas por defecto en paneles administrativos.	Acceso indebido y exposición de servicios internos.	Configurar adecuadamente los sistemas y eliminar configuraciones innecesarias.
A06	Vulnerable and Outdated Components	Uso de librerías, frameworks o software desactualizado con fallos conocidos.	Framework vulnerable explotado mediante una falla pública.	Ejecución remota de código y compromiso del sistema.	Mantener componentes y dependencias actualizados.
A07	Identification and Authentication Failures	Fallos relacionados con autenticación, sesiones o manejo de credenciales.	Contraseñas débiles que permiten ataques de fuerza bruta.	Robo de cuentas y acceso no autorizado.	Implementar autenticación multifactor y políticas seguras de contraseñas.
A08	Software and Data Integrity Failures	Falta de validación en software, actualizaciones o datos recibidos.	Instalación de paquetes alterados con código malicioso.	Ejecución de malware y manipulación de información.	Verificar integridad mediante firmas digitales y fuentes confiables.
A09	Security Logging and Monitoring Failures	Ausencia de monitoreo y registros adecuados de seguridad.	Un ataque permanece sin detectarse durante semanas.	Dificulta detectar y responder a incidentes.	Implementar sistemas de monitoreo, alertas y registros de eventos.
A10	Server-Side Request Forgery (SSRF)	El servidor realiza solicitudes manipuladas hacia recursos internos o externos.	Un atacante fuerza al servidor a acceder a servicios internos privados.	Exposición de información y acceso a infraestructura interna.	Validar URLs y restringir conexiones del servidor a recursos internos.

3- Investiga al menos 2 casos reales de cada vulnerabilidad (puedes buscar en: cvedetails.com, securityfocus.com o artículos de noticias tecnologicas). Documenta el nombre de la vulnerabilidad, la empresa afectada, el año, y como fue explotada.

Vulnerabilidad	Empresa afectada	Año	Cómo fue explotada
Broken Access Control	Facebook	2019	Usuarios podían acceder a información de otras cuentas debido a fallos de permisos.
Broken Access Control	Microsoft	2021	Un error de acceso permitió visualizar información restringida en servicios cloud.
Cryptographic Failures	Equifax	2017	Datos sensibles fueron expuestos debido a mala protección y cifrado insuficiente.
Cryptographic Failures	Marriott	2018	Información de millones de clientes fue robada por protección deficiente de datos.
Injection	Sony Pictures	2011	Atacantes utilizaron SQL Injection para acceder a bases de datos y robar información.
Injection	TalkTalk	2015	Una vulnerabilidad SQL Injection permitió extraer datos de clientes.
Insecure Design	Zoom	2020	Problemas de diseño permitieron accesos no autorizados a reuniones privadas.
Insecure Design	TikTok	2022	Fallos de diseño permitían manipular enlaces y acceder a información sensible.
Security Misconfiguration	Capital One	2019	Una mala configuración en la nube permitió acceder a datos de clientes.
Security Misconfiguration	Equifax	2017	Un servidor mal configurado facilitó el ataque y la filtración de datos.
Vulnerable and Outdated Components	Apache Log4j	2021	Librerías vulnerables permitieron ejecución remota de código.
Vulnerable and Outdated Components	SolarWinds	2020	Software comprometido mediante componentes inseguros y desactualizados.
Identification and Authentication Failures	Facebook	2018	Tokens de acceso comprometidos permitieron secuestrar cuentas.
Identification and Authentication Failures	Twitter	2020	Atacantes manipularon accesos internos y tomaron control de cuentas verificadas.
Software and Data Integrity Failures	SolarWinds	2020	Actualizaciones comprometidas instalaron código malicioso en clientes.
Software and Data Integrity Failures	Codecov	2021	Scripts alterados comprometieron información de usuarios y empresas.
Security Logging and Monitoring Failures	Uber	2016	El ataque no fue detectado rápidamente por falta de monitoreo adecuado.
Security Logging and Monitoring Failures	Yahoo	2014	La empresa tardó en detectar y reportar una filtración masiva de datos.
SSRF	Capital One	2019	Atacantes explotaron SSRF para acceder a infraestructura interna en AWS.

Fuentes y videos

- NIST Vulnerability Database
- OWASP Top 10 Official
- OWASP SQL Injection
- OWASP A03 Injection
- OWASP Query Parameterization Cheat Sheet
- CVE Details
- SQL Injection OWASP Explainer (YouTube)
- (TikTok) OWASP Top 10
- (TikTok) SQL Injection
- OWASP Top 10 Explained (you tube)



4-Para al menos 3 vulnerabilidades, incluye codigo vulnerable EN PSEUDOCODIGO O JAVASCRIPT y como deberia escribirse de forma segura. Ejemplo: SQL Injection vulnerable vs parametrized query.

SQL Injection

```
1  //SQL Injection
2
3
4  //Vulnerable
5
6  const query = "SELECT * FROM users WHERE user='" + username + "'";
7
8  //Seguro
9
10 const query = "SELECT * FROM users WHERE user=?";
11 db.query(query,[username]);
```

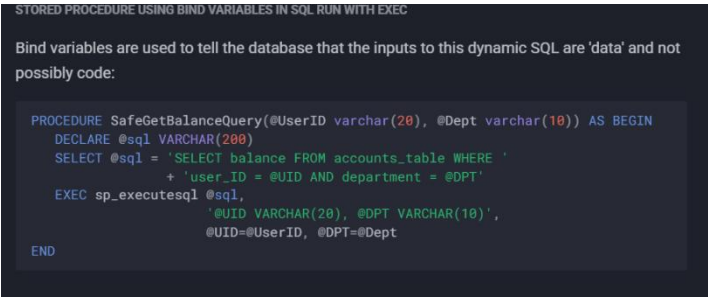
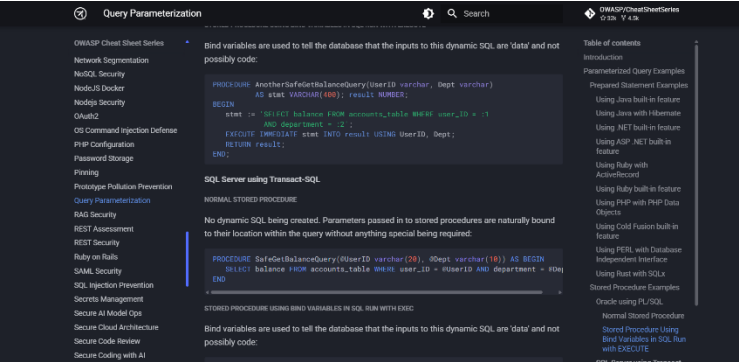
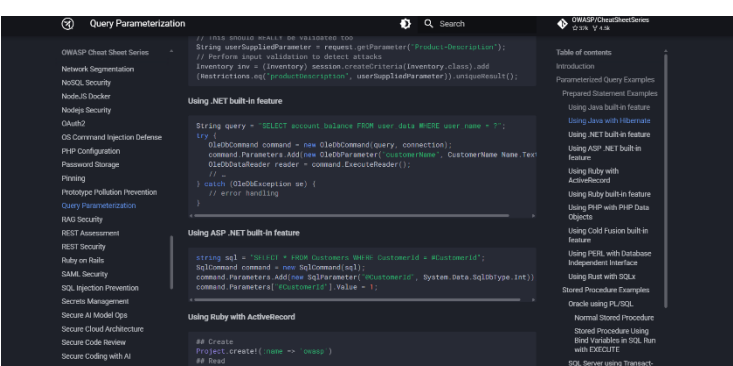
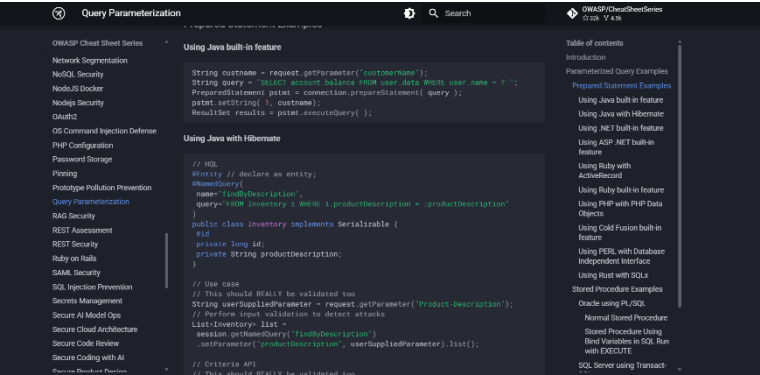
Authentication Failures

```
1  //Authentication Failures
2
3
4  //Vulnerable
5
6  if(password == "123456"){
7    login();
8  }
9
10 //Seguro
11
12 if(compareHash(password, storedHash)){
13   login();
14 }
```

Security Misconfiguration

```
1  //Security Misconfiguration
2
3
4  //Vulnerable
5
6  app.listen(3000);    Cannot find name 'app'.
7
8  //Seguro
9
10 app.listen(3000,"127.0.0.1");    Cannot find name 'app'.
```

Fuente del código seguro: [OWASP Query Parameterization Cheat Sheet](#)



Link: https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

5- Crea un ranking personal de las 5 vulnerabilidades que consideres mas criticas segun tu criterio (pueden coincidir o no con el ranking de OWASP). Justifica cada seleccion en 2-3 lineas.

Ranking	Vulnerabilidad	Justificación
1	Injection	Permite acceder y manipular bases de datos completas, causando robo masivo de información sensible.
2	Broken Access Control	Puede permitir que usuarios no autorizados accedan a funciones privadas del sistema.
3	Authentication Failures	Facilita el robo de cuentas y acceso ilegal a información personal.
4	Security Misconfiguration	Configuraciones incorrectas pueden exponer servidores y servicios completos.
5	Vulnerable and Outdated Components	Software desactualizado contiene vulnerabilidades públicas fáciles de explotar.

6- Redacta una conclusion final de minimo 200 palabras sobre la importancia de conocer el OWASP Top 10 como desarrollador y como aplicarias este conocimiento en tu practica profesional.

La lista OWASP Top 10 representa una de las herramientas más importantes dentro del área de la ciberseguridad y el desarrollo de software seguro. Conocer estas vulnerabilidades permite a los desarrolladores comprender cuáles son los errores más comunes que ponen en riesgo aplicaciones web y datos de los usuarios. Actualmente, muchas empresas dependen

completamente de sistemas digitales, por lo que un fallo de seguridad puede provocar pérdidas económicas, robo de información, daño a la reputación e incluso problemas legales.

Durante esta práctica fue posible identificar cómo vulnerabilidades como SQL Injection, fallos de autenticación o configuraciones inseguras han afectado a grandes empresas alrededor del mundo. Esto demuestra que la seguridad no debe considerarse algo opcional o secundario dentro del desarrollo de software. Desde las primeras etapas de diseño, es importante aplicar buenas prácticas de programación segura, validar entradas de usuarios, proteger contraseñas y mantener actualizados los componentes utilizados.

Como futura desarrolladora, considero que el conocimiento del OWASP Top 10 me ayudará a crear aplicaciones más seguras y confiables. También permitirá identificar riesgos antes de que sean explotados por atacantes. En proyectos futuros aplicaría medidas de seguridad como autenticación multifactor, consultas parametrizadas, monitoreo constante y validación de datos para reducir vulnerabilidades y proteger la información de los usuarios.

```
select id, firstname, lastname from authors where firstname = 'evil' and lastname = 'newman'
which the database attempts to run as:
Incorrect syntax near '!' as the database tried to execute evil!
A safe version of the above SQL statement could be coded in Java as:

String firstname = req.getParameter("firstname");
String lastname = req.getParameter("lastname");
// FIXME: do your own validation to detect attacks
String query = "SELECT id, firstname, lastname FROM authors WHERE firstname = ? and lastname = ?";
PreparedStatement pstmt = connection.prepareStatement(query);
pstmt.setString(1, firstname);
pstmt.setString(2, lastname);
try {
    ResultSet results = pstmt.executeQuery();
}

Example 2
The following C# code dynamically constructs and executes a SQL query that searches for items matching a specified name.
The query restricts the items displayed to those where owner matches the user name of the currently-authenticated user.

...
string username = ctx.getAuthenticatedUser();
string query = "SELECT * FROM items WHERE owner = '"
    + username + "' AND itemname = '"
    + itemname.Text + "'";
```

What is Injection and why it ranks top of OWASP Top 10 vulnerabilities?

Injection sits comfortably on the top of the OWASP Top 10 vulnerabilities for the last decade. This is for a good reason. In fact, injection is a broad class of vulnerabilities that you can find on pretty much any target. Let's take the definition of the OWASP Top 10 for injection and analyze it:

Injection flaws, such as SQL, NoSQL, OS, and LDAP injection, occur when untrusted data is sent to an interpreter as part of a command or a query. The attacker's hostile data can trick the interpreter into performing unintended actions.

I highlighted the key ideas in *italics*. The first thing to notice is that injection is not specific to a technology. In fact, any feature which expects and processes input is potentially vulnerable to injection.

The second thing to point out is how large the attack surface is. Tell me how many features you encountered which fall under this very scenario? I'd say most of them. In fact, even a simple search feature on a website takes your input, uses it as part of a command, queries a data store and returns the results to you.

Continuing on the example above, a malicious user can inject a malicious input, called the *payload*, to perform unintended results by the vulnerable system. If successful, the malicious user can trick the application into returning sensitive information, modify data or delete it altogether.



SQL injection explained – OWASP Top 10 vulnerabilities

April 22, 2021 by thehackerish

Welcome to this new episode of the OWASP Top 10 vulnerabilities course, where we explain in detail each vulnerability. In this blog post, you will learn SQL injection.

This is a subset of the OWASP Top 10 injection vulnerabilities. If you'd like to have a bigger picture of injection, I invite you to [read this blog post](#) before continuing.

What is SQL injection?

This vulnerability happens where the application processes malicious user input to query a SQL database. A n example would be the login feature of an admin portal. In fact, a vulnerable piece of code could be as follows:

```
userEmail = getParam("emailParameter");
userPass = getParam("passParameter");
sqlQuery = "SELECT * FROM users WHERE email = '" + userEmail + "' AND
pass = '" + userPass + "'";
if sqlQuery{
    accessAdminDashboard();
}
```